Heart Disease Prediction: End-to-End MLOps Pipeline

Machine Learning Operations (AIMLCZG523) - Assignment 1

Item	Details
Name / Student ID	Ashish Jain / 2025ae05589 (email: 2025ae05589@wilp.bits-pilani.ac.in)
Code repository	https://github.com/2025ae05589/heart-disease-mlops
Lab machine folder	/home/s2025ae05589/workspace/Assignment_1/heart-disease-mlops
Deployed API (GKE)	http://34.100.139.2/docs (interactive page; /predict for JSON)
Screenshots	screenshots/ folder - 72 screenshots, named by task
Video	report/video/MLOps_Assignment1_Video_2025ae05589.mp4 (GitHub Repository) MLOps_Assignment1_Video_2025ae05589.mp4 (Google Drive)

1. Summary

A machine learning classifier has been developed to predict the presence of heart disease from patient health data, using the Cleveland data of the UCI Heart Disease dataset (303 patients, 13 features). Four models have been tuned with 10-fold cross-validation, and every run has been tracked in MLflow. Logistic Regression (C = 0.1) has been selected by a rule fixed in advance, achieving a cross-validation ROC-AUC of 0.905 and, on 61 unseen test patients, a ROC-AUC of 0.965 with a recall of 0.929.

Training has been implemented as a seven-step pipeline with two quality gates. The selected model is registered in the MLflow model registry as a challenger and is promoted to champion only when both gates are passed. The champion is exported and packaged in a Docker image with a FastAPI service. A GitHub Actions workflow of six jobs performs linting, unit testing (94 tests), training, registration, image build and testing, and deployment through a Helm chart to a GKE Autopilot cluster behind a load balancer, and the same chart is used on Minikube. Each request is logged in structured JSON, which can be searched in Cloud Logging (Logs Explorer) on GKE, and Prometheus and Grafana are used for monitoring.

The final run has been carried out from a fresh clone on the BITS virtual lab machine. The screenshots have been taken on the lab machine and on the GitHub and Google Cloud web pages. Identical results have been obtained in the notebook, in two pipeline runs and in GitHub Actions (Section 7).

2. Architecture

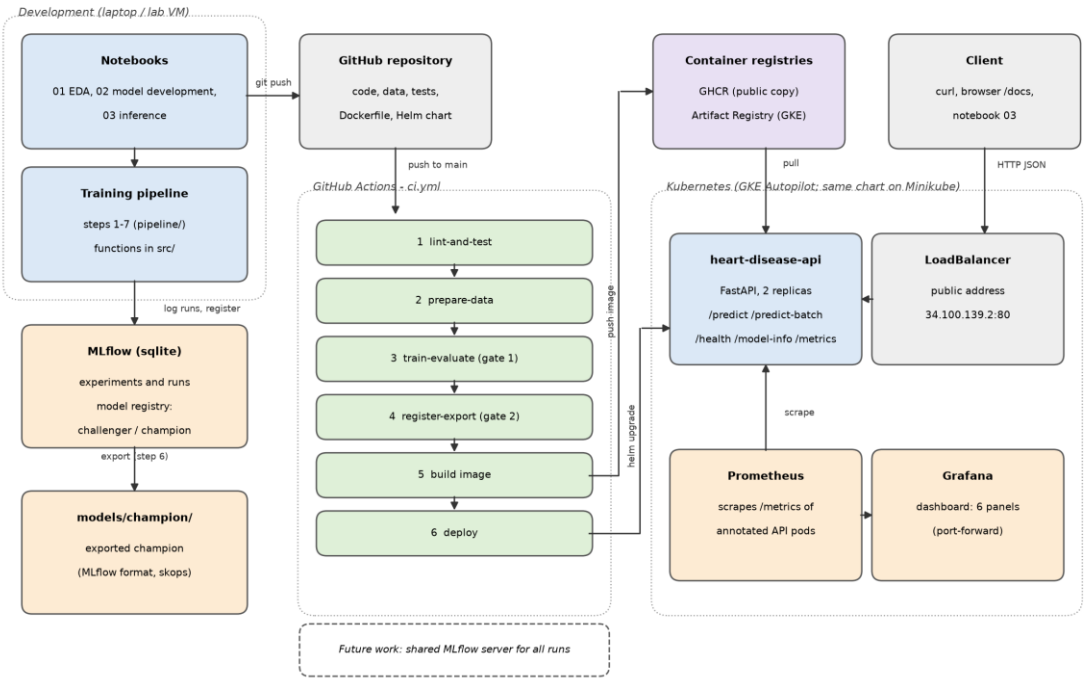


Figure 1. Architecture of the solution

The notebooks are used for analysis and model development, while the training pipeline calls the common functions in src/, ensuring that the notebooks, the pipeline and the API share the same code. The term pipeline is used in three senses: the model pipeline (preprocessing and model saved as one scikit-learn object), the training pipeline (steps 1 to 7) and the CI/CD pipeline (GitHub Actions). The key design decisions are summarised below and explained in the sections that follow.

Area	Decision	Reason (section)
Data	Cleveland file only	297 of 303 rows complete; the other files 2 of 617 (4)
Missing values	imputed inside the model pipeline	learnt from training rows only; the API applies the same rule (4, 9)
Model selection	within 0.01 ROC-AUC of the best, highest recall selected	a missed patient with disease is the costliest error; fixed before testing (5)
Quality gates	gate 1: test ROC-AUC >= 0.80; gate 2: must beat the champion	a weak model is not registered; the champion is replaced only by a better model (5, 6)
Registration	performed only by the pipeline; the API serves alias champion	a single controlled path to production (6, Appendix A)
Packaging	MLflow format with a skops file, baked into the image	no MLflow store needed in the container; skops is safer than pickle (7, 9)
CI/CD	six dependent jobs; image tagged with the commit ID	any failure stops the release; the running code is always traceable (8, 10)
Cloud access	Workload Identity Federation	no cloud key is stored (10)
Registries	Artifact Registry for GKE, GHCR for the public copy	no download charge within Google Cloud; free public access (10)
Logging	one structured JSON line per request to stdout	read by Cloud Logging without an agent; one request ID across logs (11)
New data	detected by a weekly check, accepted after review	a changed file does not prove that the data is valid (8, Appendix A)

3. Setup and Installation

Python 3.12 and Git are required, and Docker, Minikube and Helm are required for deployment.

```
git clone https://github.com/2025ae05589/heart-disease-mlops.git
cd heart-disease-mlops
python -m venv .venv && source .venv/bin/activate   # Windows: .venv\Scripts\activate
pip install -r requirements.txt
ruff check . && pytest -v                 # lint and 94 unit tests
python -m pipeline.step1_download_data    # optional: download, check the UCI file hashes
python -m pipeline.run_all                # steps 2-7: prepare, train, evaluate, register, export
mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5000    # view the runs

minikube start && eval $(minikube docker-env)
# option 1: build the image and deploy it with Helm
docker build -t heart-disease-api:local .
helm upgrade --install heart-disease-api deployment/helm/heart-disease-api \
  --set image.repository=heart-disease-api --set image.tag=local --wait
kubectl port-forward svc/heart-disease-api 8000:80
curl -X POST http://127.0.0.1:8000/predict -H "Content-Type: application/json" \
  -d @api/sample_request.json

# option 2: run the published image without building it (no login needed)
IMG=ghcr.io/2025ae05589/heart-disease-api:e86d176d6151e2a722380f9b6a991cea3a32bed5
docker run --rm -p 8000:8000 $IMG
curl http://$(minikube ip):8000/model-info    # run a4872b0f, the same model as GKE
```

On the lab machine, the steps have been executed in the following order: notebook 01, notebook 02, the first pipeline run (version 1 promoted to champion), the Docker build, deployment on Minikube with Helm, notebook 03 (comparison of the exported model with the API), a second pipeline run to test gate 2, and finally a push to GitHub. The live API on GKE can be accessed without any setup at http://34.100.139.2/docs.

Supporting screenshots (click to open): 00 lab session (BITS virtual lab) · 01 clone and setup · 01b repository files · 02a lint and tests · 02b 94 tests passed · 03 data download and hash check · 08 pipeline run 1

4. Data Acquisition and EDA

4.1 Data source and cleaning

The four processed files of the UCI dataset (Cleveland, Hungarian, Switzerland and VA) are downloaded by step 1, and each file is verified against a recorded SHA-256 hash. Only the Cleveland file has been used, as it contains 297 complete rows out of 303, whereas the other three files together contain only 2 complete rows out of 617, and the Switzerland file records cholesterol as 0 for all patients.

The cleaning and quality checks have produced the following results.

- No duplicates, invalid codes or impossible values have been found. These checks are repeated as unit tests, so that a faulty data file fails CI.
- Six values are missing (four in ca and two in thal). They are retained in the cleaned file and imputed inside the model pipeline with the most frequent value of the training rows, so that nothing is learnt from the test rows and the API applies the same rule.
- The target (0 to 4) has been converted to a binary target, giving nearly balanced classes: 164 patients (54%) without and 139 (46%) with disease.
- Twenty outlier values (IQR rule) in 19 patients have been retained, as the values are plausible and 68% of these patients have the disease, compared with 46% of all patients.

4.2 Main findings

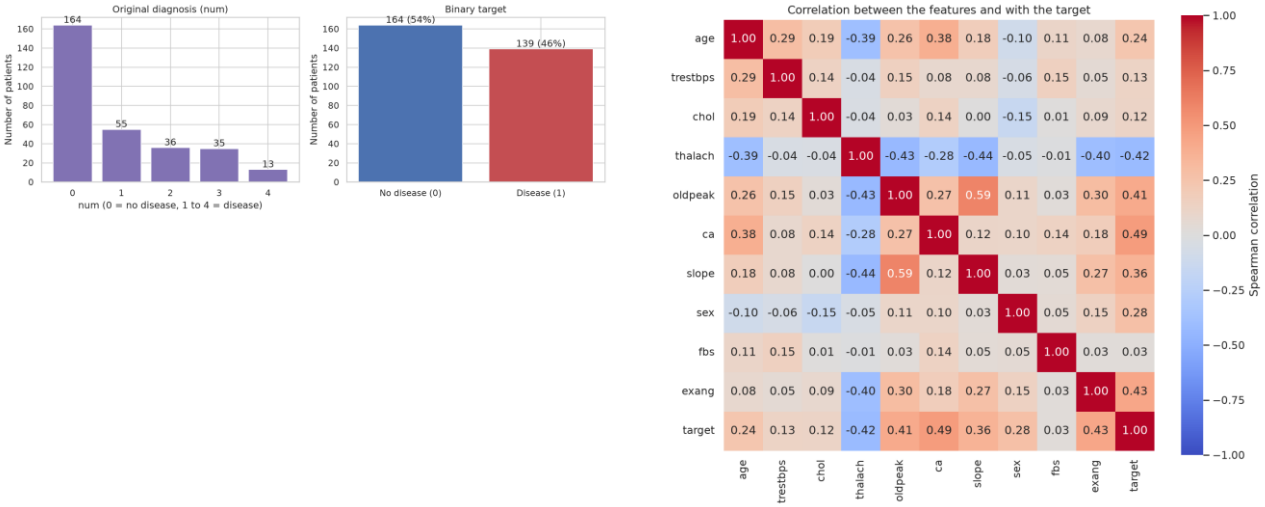


Figure 2. Class balance (left) and Spearman correlation (right) (screenshot 04 · 05)

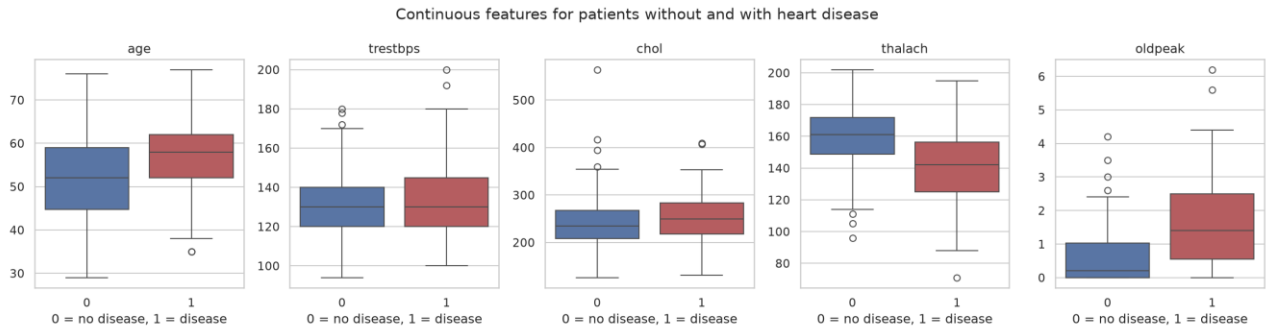


Figure 3. Continuous features for patients without and with heart disease

The main findings of the exploratory analysis are summarised below.

- Among the continuous features, thalach and oldpeak separate the two groups best (Figure 3) and have the strongest correlation with the target (−0.42 and 0.41, Figure 2). Patients with disease have a lower mean thalach (139 against 158) and a higher mean oldpeak (1.6 against 0.6), while trestbps and chol differ only slightly (correlation 0.13 and 0.12).

- Among the categorical features, ca, cp, thal and exang show a strong relation with the target, whereas fbs shows almost none. In Figure 2, ca and exang have a correlation of 0.49 and 0.43, against 0.03 for fbs. The features cp and thal are nominal and are not included in Figure 2, but the disease share is 73% for cp = 4 against 18% to 30% for the other values, and 67% and 76% for thal = 6 and 7 against 22% for thal = 3 (notebook 01, section 11.2, chart).
- No two features are strongly correlated. The highest correlation is 0.59 (oldpeak and slope, Figure 2), well below the level of about 0.8 at which two features carry the same information (multicollinearity), so none has been removed.
- The disease share does not rise steadily with the category codes (for slope, 25%, 65% and 57% for 1, 2 and 3), so all categorical features, including ca, have been one-hot encoded (one 0/1 column per value).
- Only 25 of the 97 women have disease, so the trained model has few such examples and may be less reliable for women (Section 12).

Histograms, category counts and disease share by category are available in notebook 01 and in report/figures/.

5. Feature Engineering and Model Development

5.1 Preprocessing

Fixed rules are applied in the cleaning code, while every step that learns from the data is placed inside the model pipeline (src/features/preprocessing.py). After encoding, the 13 features become 28 columns.

Feature group	Steps in the model pipeline
Numerical (age, trestbps, chol, thalach, oldpeak)	median imputer, standard scaler
Categorical (sex, cp, fbs, restecg, exang, slope, ca, thal)	most-frequent imputer, one-hot encoder (unknown categories ignored)

5.2 Models and tuning

The four models have been chosen on three criteria: the dataset is small (303 patients, 242 for training), every prediction must come with a confidence score, and a prediction should be explainable. Logistic Regression has been included as the simple and explainable reference. Random Forest has been included as it handles non-linear patterns, such as those seen for ca and slope, and the retained outliers. Gradient Boosting and XGBoost have been included as boosted trees, which are usually accurate on table data, with XGBoost giving more controls against overfitting. Support Vector Machine has not been included, as it does not give a probability directly, and models that need much more data, such as neural networks, have been left out.

The data has been split 80/20 (242 training and 61 test patients) with stratification and a fixed seed. Each model has been tuned by grid search with 10-fold stratified cross-validation on the training patients, scored by ROC-AUC, and the default setting of each model has been included among the combinations.

Model	Settings tried	Best setting
Logistic Regression	C: 0.01, 0.1, 1, 10 (4)	C = 0.1
Random Forest	max_depth, min_samples_leaf, n_estimators (18)	depth 5, leaf 3, 300 trees
Gradient Boosting	learning_rate, max_depth, n_estimators (18)	rate 0.1, depth 1, 50 trees
XGBoost	same grid + XGBoost default (19)	rate 0.05, depth 2, 100 trees

5.3 Results and selection

Model (cross-validation means)	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.831	0.852	0.784	0.810	0.905 ± 0.048
Random Forest	0.810	0.829	0.774	0.788	0.905 ± 0.040
Gradient Boosting	0.831	0.866	0.766	0.806	0.899 ± 0.038
XGBoost	0.797	0.807	0.758	0.773	0.903 ± 0.039

The four models lie within 0.006 ROC-AUC of each other, well inside the spread across folds (about 0.04), so ROC-AUC alone cannot separate them. A selection rule has therefore been fixed before the test set was used: models within 0.01 of the best ROC-AUC are treated as equivalent, and among them the model with the highest recall is selected, as a missed patient with disease is the more costly error. Logistic Regression and Random Forest tie at 0.905, and **Logistic Regression has been selected for its higher recall (0.784 against 0.774)**. It is also the simplest model to explain.

5.4 Test set and quality gate

On the 61 test patients, the selected model has achieved an accuracy of 0.885, precision of 0.839, recall of 0.929, F1 of 0.881 and ROC-AUC of 0.965, identifying 26 of the 28 patients with disease. Gate 1 has been fixed in advance: the test ROC-AUC must be at least 0.80, about two standard deviations below the cross-validation mean. It serves as a floor to catch a faulty model rather than as a target. On failure, the step ends with exit code 1 and CI is stopped. The test results exceed the cross-validation means, but with 61 patients a single patient changes recall by 0.036, so the cross-validation means remain the more reliable estimate. The test set has not been used for any choice: the model, its settings and gate 1 were fixed before it was evaluated, and nothing has been changed after the result was seen.

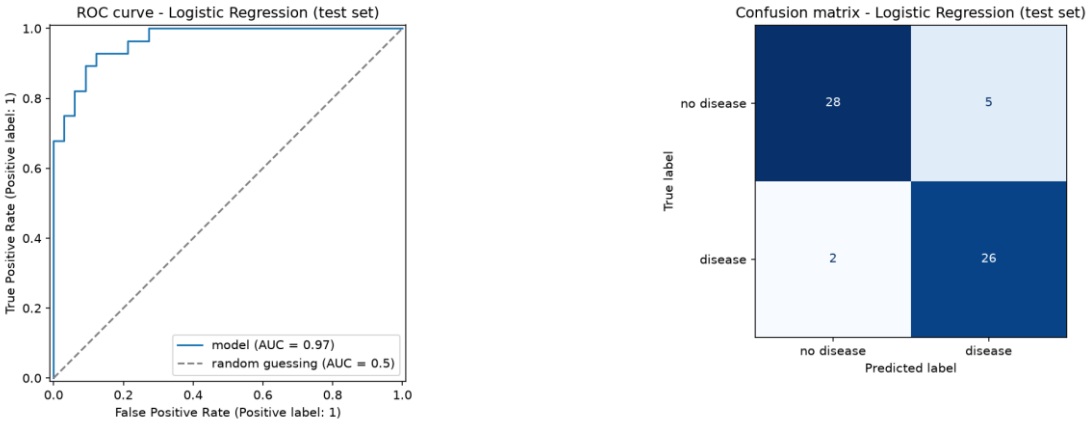


Figure 4. ROC curve and confusion matrix of the selected model on the test set (screenshot 07 · 07a)

Supporting screenshots (click to open): 06 model comparison

6. Experiment Tracking

MLflow has been integrated with a SQLite store and a local artifact folder. The tracking address can be changed through the MLFLOW_TRACKING_URI variable without code changes, and this has been verified with a local MLflow server.

The following information is logged for every run.

- **Runs:** one parent run is created per model and one child run per combination of settings, giving 59 child runs. Notebook runs are tagged as experiments, so they are never selected by the pipeline.
- **Parameters:** the model type, settings, number of folds, random state, tuning metric, data file paths with their SHA-256 hashes and the number of training patients.
- **Metrics:** the mean and standard deviation of accuracy, precision, recall, F1 and ROC-AUC. Test metrics are added to the selected run.
- **Artifacts:** ROC curves, confusion matrices, the table of all combinations and the model with its input example and package list.

The selected model is registered as heart-disease-classifier with the alias challenger, and is promoted to champion only if its test ROC-AUC is above that of the current champion (gate 2). Otherwise, it is tagged failed_gate. Each version carries the git commit and the data hash, and the API always loads the alias champion rather than a fixed version number. This champion-challenger approach, in which a challenger is compared with the champion on held-out data before promotion, is a common MLOps practice (Databricks, MLOps workflows) and is supported by MLflow model aliases.

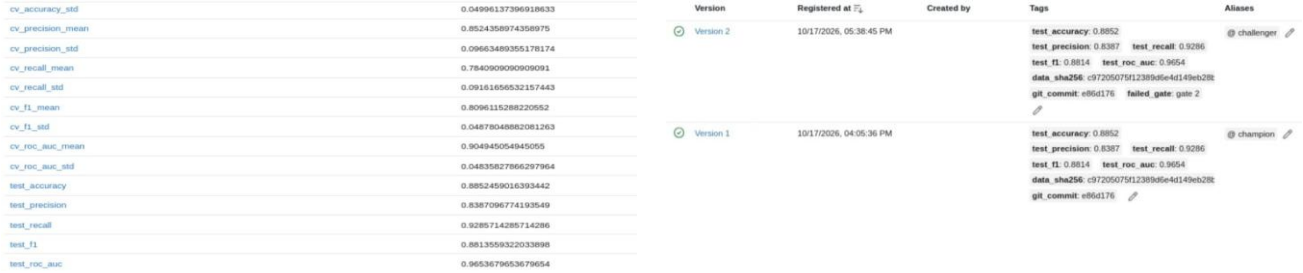


Figure 5. Champion run c08938bd: test metrics (left); registry with version 1 champion and version 2 challenger (right); cropped, the full screenshot opens from the link (screenshot 21a · 23d)

Gate 2 has been tested on the lab machine by running the pipeline a second time on the same data. The run produced the same model (Logistic Regression, C = 0.1, run ff3eaf25), registered as version 2, with the same test ROC-AUC of 0.965. As gate 2 requires a higher score than the champion, version 2 has not been promoted: it has kept the challenger alias with the tag failed_gate = gate 2, and version 1 has remained the champion served by the API. The runs, plots and registry history are shown in the screenshots linked below.

Supporting screenshots (click to open): 19 pipeline run 2, gate 2 · 20 MLflow runs · 21b champion run parameters · 21c logged model · 22a CV confusion matrix · 22b CV ROC curve · 22c test confusion matrix · 22d test ROC curve · 23a registry aliases · 23b registry version 2 · 23c registry version 1 · 21d run 2 metrics · 21e run 2 parameters

7. Model Packaging and Reproducibility

Step	Module (pipeline/)	What it does
1	step1_download_data	download the UCI files, check hashes (run on its own)
2	step2_prepare_data	clean the data, binary target, save heart_clean.csv
3	step3_train_models	tune the four models, log every run to MLflow
4	step4_evaluate_model	select by the rule, evaluate on the test set, gate 1
5	step5_register_model	register as challenger, gate 2, alias champion
6	step6_export_model	export the champion to models/champion/
7	step7_validate_model	load the exported folder alone and predict 5 patients

The final model has been saved in MLflow format, with the preprocessing and the classifier stored as one object in the skops format, which is safer to load than pickle. All package versions are pinned in requirements.txt, and a clean installation has been tested. The exported model carries its own package list, which is installed in the Docker image. All settings are kept in src/config.py and random seeds are fixed. Reproducibility has been verified across four runs on the lab machine and the GitHub runner, as summarised below.

Where	MLflow run	CV ROC-AUC (LR / RF / GB / XGB)	Selected	Test ROC-AUC, recall
Notebook 02, lab machine	notebook runs	0.905 / 0.905 / 0.899 / 0.903	LR, C = 0.1	0.965, 0.929
Pipeline run 1, lab machine	c08938bd	same	same	same (version 1, champion)
Pipeline run 2, lab machine	ff3eaf25	same	same	same (version 2, gate 2)
CI run #19, GitHub runner	a4872b0f	same	same	same (served on GKE)

The sample patient has also received the same prediction everywhere (disease, confidence 0.6275) in notebook 03, on Minikube, in the CI container test and on GKE. The run IDs differ only because each environment keeps its own MLflow store (Section 12).

Supporting screenshots (click to open): 15 notebook 03 vs API · 24c CI training · 24d CI evaluation, gate 1 · 29 GKE model-info

8. CI/CD Pipeline and Automated Testing

A GitHub Actions workflow has been created that runs on every push to main and can also be started manually. Each job starts only after the previous one has passed, so a lint error, a failed test or a failed gate stops the release, and the job log shows the cause. Pushes that change only Markdown files, notebooks, the report or screenshots are skipped, as these do not affect the trained model or the API. Run #19 for commit e86d176 completed in 14 min 47 s and stored 10 artifacts.

Job	What it does	Artifacts
1 lint-and-test	ruff and 94 pytest tests	unit-test-report
2 prepare-data	step 2	prepare-data-log, clean-data
3 train-evaluate	steps 3 and 4 (gate 1)	train-evaluate-log, trained-models
4 register-export	steps 5 to 7 (gate 2, export, check)	register-export-log, champion-model, mlflow-store
5 build	build the image, start it, send a valid, an invalid (422) and a metrics request, push to GHCR and Artifact Registry	image-check
6 deploy	Helm on GKE, Prometheus and Grafana, check the live API and that Prometheus reads the pods	deploy-check



Figure 6. CI run #19: six jobs passed (left); 94 tests passed in the lint-and-test job (right); cropped, the full screenshot opens from the link (screenshot 24 · 25)

Test file	Tests	What is checked
test_data_quality.py	17	codes, value ranges, duplicates of the clean file
test_data_preparation.py	8	cleaning, target, split sizes and stratification
test_definitions.py	17	model definitions and grids
test_preprocessing.py, test_tuning.py	6	pipeline steps, cross-validation
test_model_training.py, test_evaluation.py	10	probabilities, selection rule, both gates
test_api.py	27	endpoints, validation (422), batch, missing values
test_monitoring.py	9	metrics and log lines

A second workflow downloads the UCI files every Monday and compares their hashes, so that a change in the source data is detected. It commits nothing: new data is accepted after review, and that commit triggers the main workflow, which retrains the model (Appendix A). As the UCI files are fixed, this serves as a demonstration of the check. The log of each job in run #19 has been captured, from linting to deployment.

Supporting screenshots (click to open): 24i CI job graph · 25a CI lint · 25b CI tests start · 24a CI artifacts · 24b CI prepare-data · 24e CI register, export, check · 24f CI image build · 24g CI container test · 24h CI container metrics

9. Model Containerization

Endpoint	Method	Purpose
/predict	POST	prediction, label and confidence for one patient
/predict-batch	POST	up to 100 patients, checked row by row
/health	GET	whether the model is loaded (Kubernetes probes)
/model-info	GET	model name, version, MLflow run ID, training time
/metrics	GET	metrics for Prometheus

A Docker image has been built for the FastAPI serving API from python:3.12-slim.

- The image installs the model's package list and the API packages, copies the exported model and runs as a non-root user.
- The request schema requires all 13 features with valid codes and ranges. Only ca and thal may be null, in which case they are imputed by the model pipeline and listed in filled_fields.
- An invalid request is rejected with status 422 and the list of invalid fields.
- The confidence is the probability of the predicted class, with a threshold of 0.5.
- The image is 911 MB on the lab machine (about 278 MB compressed in the registry), a large part of which is the MLflow package used only to load the model (Section 12).

```
POST /predict  (api/sample_request.json)
{"prediction": 1, "label": "disease", "confidence": 0.6275, "filled_fields": [], "model_version": "1"}
POST /predict  (same patient, ca and thal null)
{"prediction": 0, "label": "no disease", "confidence": 0.6142, "filled_fields": ["ca", "thal"],
"model_version": "1"}
```



Figure 7. Image build on the lab machine (left); /docs request and response on Minikube (right); cropped, the full screenshot opens from the link (screenshot 10 · 13)

Supporting screenshots (click to open): 12 curl requests · 12a batch and metrics · 24g CI container test

10. Production Deployment

The API has been deployed using the same Helm chart on both Minikube and GKE. The chart creates a Deployment with two replicas, startup, readiness and liveness probes on /health and resource limits, together with a Service of type LoadBalancer on port 80. The image tag must be supplied at install time, and CI uses the commit ID, so the running code is always traceable.

- **GKE:** an Autopilot cluster in asia-south1 serves the public API at http://34.100.139.2, and a rolling update replaces the pods one at a time.
- **Cloud access:** the deploy job authenticates to Google Cloud through Workload Identity Federation, a trust between the repository and the project, so no key is stored.
- **Registries:** the image is kept in Artifact Registry, which is private and in the same region as the cluster, and in GHCR, which holds the public copy.
- **The public GHCR image** has been pulled on the lab machine without a login and has served the same model as GKE (run a4872b0f), as shown in screenshots 27a and 27b.
- **Minikube:** on the lab machine, the image is built inside Minikube and installed with the same chart, and the API is reached through kubectl port-forward.
- **Verification:** health, model information, single and batch predictions, a rejected request and the metrics endpoint have been verified on Minikube and GKE, and the main checks are repeated by the build and deploy jobs on every release.



Figure 8. Minikube pods and service on the lab machine (left); GKE services with the external load balancer (right); cropped, the full screenshot opens from the link (screenshot 11 · 32)

Two models are in service by design: GKE runs the model trained by CI (run a4872b0f), and Minikube runs the model trained on the lab machine (run c08938bd). They are the same model with the same results (Section 7).

Supporting screenshots (click to open): 10a Minikube start · 26 CI deploy to GKE · 26a CI public address check · 27 GHCR package · 27a GHCR pull test · 27b GHCR model check · 28 GKE /docs predict · 28a GKE curl requests · 28b GKE batch · 29 GKE model-info · 30 GKE cluster · 31 GKE workloads · 33 Artifact Registry images · 33a Artifact Registry repo · 34 Cloud Shell kubectl, helm, logs · 34a Cloud Shell kubectl

11. Monitoring and Logging

Structured JSON logging has been integrated into the API to capture request timestamps, severity, HTTP status codes, response latency, request IDs, model version information and prediction outcomes. Validation errors and batch prediction results are also recorded, with one summary line per batch and one line per row. Health and metrics calls are excluded. A log line from the GKE deployment is shown below.

```
{"time": "2026-10-17T16:42:46Z", "severity": "INFO", "message": "POST /predict status=200
```

```
time_ms=43.5 request_id=b8ca6577 model_version=1 prediction=0 confidence=0.6142
input={\"age\":48.0, ... \"ca\":null,\"thal\":null}"}
```

The logs are written to stdout in JSON so that Google Cloud Logging reads the severity directly, without an extra agent. The request ID is also returned in the X-Request-ID header, enabling a response to be matched to its log line. The application logs have been inspected using Kubernetes pod logs on Minikube and Google Cloud Log Explorer for the GKE deployment, where the JSON fields are parsed and can be filtered. On GKE, Cloud Logging provides log inspection and filtering without a separate Loki deployment.

Prometheus and Grafana have been installed on both Minikube and GKE for metrics collection and visualization, enabling monitoring of API activity and performance.

- Six metrics are exposed on /metrics, three for the service (requests by path and status, response time, model loaded) and three for the model (predictions by class, confidence, missing fields).
- Prometheus discovers the API pods through pod annotations and scrapes them every 15 seconds.
- Grafana loads a six-panel dashboard from the repository, showing requests and errors per minute, median and 95th-percentile response time, copies with the model loaded, predictions per minute by class and the spread of confidence. Missing fields are seen on /metrics only.
- Within days of going public, 272 requests to unknown paths were received from internet scanners. As all unknown paths are counted under a single label, the metrics remained compact.

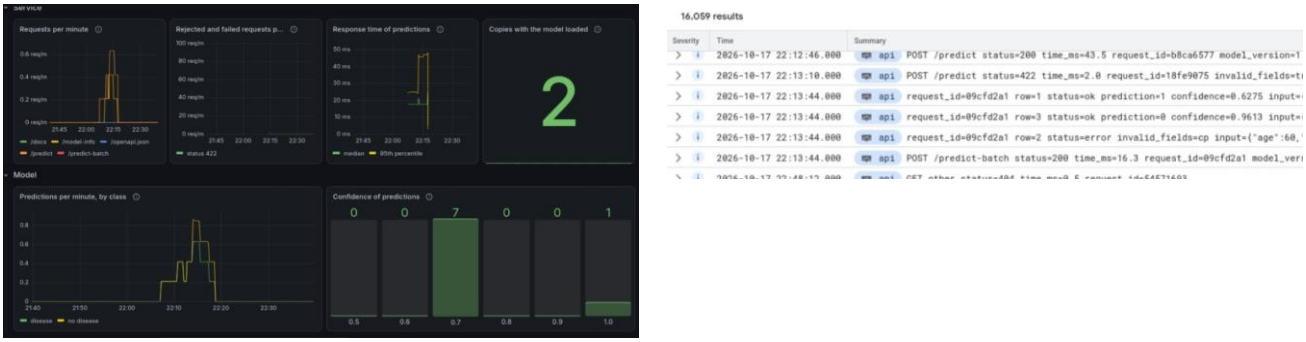


Figure 9. Grafana dashboard on GKE (left); API log lines in Cloud Logging (right); cropped, the full screenshot opens from the link (screenshot 35 · 37)

Screenshots demonstrate application logging, cloud log inspection, Prometheus targets and API metrics monitoring.

Supporting screenshots (click to open): 14 API logs (Minikube) · 16 monitoring pods · 17 Grafana (Minikube) · 18 /metrics endpoint · 28c GKE metrics · 36 Prometheus targets (GKE) · 37a Cloud Logging entry

12. Limitations and Future Work

Evaluation and promotion

- **Limitation:** Gate 2 compares each challenger with the champion on the same 61 test patients, on one metric (ROC-AUC) and without a margin. As more challengers are compared on these patients, one may win by chance (adaptive overfitting to the test set), and the small size makes the scores less stable.
- **Reason:** The cause is the small, fixed dataset, not the champion-challenger design itself. No newly labelled patients are available, and with only 303 patients, setting aside a second test set would reduce the training data further.
- **Future mitigation:** The challenger and the champion would both be evaluated on the same newly labelled patients, with a predefined margin and a recall check. A shadow or canary comparison on live traffic would precede the switch, and a separate test set would be reserved for the final evaluation.

Tracking and data

- **Separate MLflow stores:** MLflow is not shared, as the laptop, the lab machine and each CI run keep their own store. Gate 2 is therefore effective only where the store is retained (the lab machine), and CI always shows version 1. A shared MLflow server is the first planned improvement (Figure 1).

- **Small subgroups and weak features:** Results for women rest on 97 patients, of whom 25 have disease, and fbs has been retained despite its weak relation with the target. Reporting results by sex and a feature importance check are planned as future work.

Production and operations

- **No accuracy in production:** Accuracy cannot be measured in production, because the confirmed diagnosis of a patient is not sent back to the API. Only indirect signals are monitored: the missing fields, which describe the input, and the share of disease predictions and the confidence, which describe the model's output. These can hint at a change in the incoming patients but cannot show a drop in accuracy. In future, confirmed diagnoses could be matched to the logged predictions by request ID to track recall and accuracy, and the input values could be compared with the training data to detect drift.
- **Open public API:** The public API has no authentication or rate limiting, and the scanner traffic illustrates this exposure. Real use would require appropriate access controls and rate limiting, for example through an API gateway.
- **Patient data in logs:** Each log line contains the full patient input so that predictions can be verified in this assignment. The inputs come from the public, anonymised UCI dataset, and no real patient data has been used. As patient data is sensitive, a real system would not log it, or would mask it, and would restrict log access and retention.
- **Image size and housekeeping:** The image carries the full MLflow package only to load the model. It could be made smaller by opening the saved model file with the small skops library instead, so that MLflow is no longer needed in the image. The notebooks are not covered by the lint step, and all resources use the default namespace.

Appendix A. Design reasons in detail

A.1 Why the notebook does not register the model

The notebook evaluates and reports, whereas the pipeline evaluates, enforces the gates and registers. A gate cell in a notebook can be skipped and its result depends on which cells were run, whereas the pipeline runs the same committed code locally and in CI, and each registered version records its git commit and data hash. This matches the common practice of automating the training pipeline rather than a single trained model (Google, MLOps level 1).

A.2 Why new data is not committed and trained automatically

A change is detected by the weekly check, and a commit of new data triggers retraining, so only the acceptance step is manual. The reasons are given below.

- A changed hash indicates only that the file has changed, not that the data is valid, as it may be corrupt, in a new format or an error page.
- New data may require a decision, such as allowing missing values in a new field.
- The gates check model scores, not data quality, and with new data the test set changes, so gate 2 would compare scores on different patients.
- The pipeline ends with deployment to GKE, so a faulty data file could reach the public API.
- An automatic commit would need write access and a stored personal token, whereas the workflows are kept read-only and keyless.

In future, the check could open a pull request with a data validation report, and larger data could be versioned with DVC.

Appendix B. References

- Janosi, A., Steinbrunn, W., Pfisterer, M. and Detrano, R. (1989). Heart Disease. UCI Machine Learning Repository. https://doi.org/10.24432/C52P4X
- Google Cloud. MLOps level 1. https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning
- Databricks. MLOps workflows on Databricks. https://docs.databricks.com/aws/en/machine-learning/mlops/mlops-workflow
- Documentation of MLflow, FastAPI and the community Prometheus and Grafana Helm charts. Course lab manuals and webinars, AIMLCZG523, BITS Pilani WILP.